

Thread API

How to create new threads?

```
#include <pthread.h>
int
pthread_create(pthread_t *thread,
               const pthread_attr_t *attr,
               void *(*start_routine) (void*),
               void *arg);
```

→ header file required to use POSIX threads
(pthread.h)

it contains the
declarations of
functions, types,
etc. for thread
management.

without this, pthread_create and related thread functions
won't be recognized by the compiler.

is a function that creates a new thread in your program
* return type: int (returns 0 on success)

let us break down the above declaration carefully. There are
four arguments: thread, attr, start_routine, arg.

(1) `pthread_t *thread`

— pointer to thread ID (a thread handle of type `pthread_t`)

purpose: after the thread is created, this variable stores the thread identifier, which can be used later for joining the thread, canceling, etc..

eg!:-

```
pthread_t tid; // Thread ID  
pthread_create(&tid, NULL, myFunction, NULL);
```

↳ passes the address of `tid` to `pthread_create`.

(2) `const pthread_attr_t *attr`

— pointer to thread attributes.

purpose: specifies thread attributes like stack size, detach state, scheduling policy, etc.

If `NULL` is passed → default thread attributes are used.
(as in the example given above)

But if you want custom attributes, say, detach state changed, then an example would be as follows:

no `pthread_join` is possible.
when it finishes, the system automatically reclaims its resources.

the initialization guarantees that all unspecified attributes fall back to defaults.

eg:-

```
pthread_attr_t attr;  
pthread_attr_init(&attr); // required  
pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);  
pthread_create(&tid, &attr, myFunction, NULL);  
pthread_attr_destroy(&attr);
```

↳ cleans up the internal resources used by the attribute object.

(3) `void* (*start_routine) (void*)`

- pointer to a function that takes `void*` and returns `void*`

Purpose: this is the function the new thread will execute.

Note that it must match the signature

`void* function-name (void* arg)`

eg:-

```
void* myFunction(void* arg) {  
    int *num = (int*)arg;  
    printf("Hello from thread! Number = %d\n", *num);  
    return NULL;  
}
```

- Here, `myFunction` will run in the new thread.

(4) void *arg (generic pointer)

Purpose: Argument passed to start-routine, can be any data type cast to void *

eg:-

```
int number = 42;
```

```
pthread_create(&tid, NULL, myFunction, &number);
```

&number is passed to myFunction as arg. Inside myFunction, it's cast back to int *.

```
#include <stdio.h>
#include <pthread.h>

void* myFunction(void* arg) {
    int *num = (int*)arg;    // Cast void* back to int*
    printf("Hello from thread! Number = %d\n", *num);
    return NULL;
}

int main() {
    pthread_t tid;           // Thread ID
    int number = 42;

    // Create a new thread
    int result = pthread_create(&tid, NULL, myFunction, &number);
    if(result != 0) {
        printf("Thread creation failed!\n");
        return 1;
    }

    // Wait for the thread to finish
    pthread_join(tid, NULL);
    printf("Thread finished execution.\n");
    return 0;
}
```

→ a simple example
after putting everything
together.

An example to create thread that is passed two arguments, packaged into a single type

```
1  #include <stdio.h>
2  #include <pthread.h>
3
4  typedef struct {
5      int a;
6      int b;
7  } myarg_t;
```

} defines a custom struct `myarg_t`
with two integers → type of data we
want to pass to the thread.

```
8
9  void *mythread(void *arg) {
10     myarg_t *args = (myarg_t *) arg;
11     printf("%d %d\n", args->a, args->b);
12     return NULL;
13 }
```

} start routine.
accepts a
generic pointer,
inside, cast it back to `myarg_t*` to access
fields.

```
14
15  int main(int argc, char *argv[]) {
16     pthread_t p; → handle for new thread
17     myarg_t args = { 10, 20 }; → create a struct with values
18
19     int rc = pthread_create(&p, NULL, mythread, &args);
20     ... → creates new thread
21 }
```

what happens if you want to wait for a thread to complete?
↳ call the routine `pthread_join()`.

```
int pthread_join(pthread_t thread, void **value_ptr);
```

↳ waits for a thread to finish and optionally collect its return value.

Arguments

① thread of type `pthread_t`
→ the ID of the thread you want to wait for.

② value_ptr of type `void**`
→ pointer to a pointer that will store the thread's return value

(this can be NULL if you don't care about the return value)

Note that `pthread_join` is important for reclaiming resources of the terminated thread (avoids memory leaks).

A simple example to illustrate Pthread_join()

```
void* mythread(void* arg) {  
    int* result = malloc(sizeof(int));  
    *result = 42;  
    return result; // return as void*  
}  
  
int main() {  
    pthread_t tid;  
    pthread_create(&tid, NULL, mythread, NULL);  
  
    void* ret;  
    pthread_join(tid, &ret); // wait for thread, get return value  
    int* val = (int*)ret;  
    printf("Thread returned: %d\n", *val);  
    free(val);  
}
```

Why `malloc()` is used when returning a value from the thread?
Of defined as follows:

```
void* mythread(void* arg) {  
    int result = 42;  
    return &result;  
}
```

- `result` lives on the thread's stack, after the thread ends, that stack frame is destroyed. So, using it in `Pthread_join` leads to undefined behavior.

On The other hand, by using `malloc()` as below, it allocates memory for result on the heap, which survives after the thread ends. So, `Pthread_join` can safely receive this pointer. Later, the main thread can use the value and free the memory.

```
void* mythread(void* arg) {  
    int* result = malloc(sizeof(int));  
    *result = 42;  
    return result;  
}
```

A slightly complicated example for `pthread_join`

```
1  typedef struct { int a; int b; } myarg_t;  
2  typedef struct { int x; int y; } myret_t;  
3  
4  void *mythread(void *arg) {  
5      myret_t *rvals = Malloc(sizeof(myret_t));  
6      rvals->x = 1;  
7      rvals->y = 2;  
8      return (void *) rvals;  
9  }  
10  
11 int main(int argc, char *argv[]) {  
12     pthread_t p;  
13     myret_t *rvals;  
14     myarg_t args = { 10, 20 };  
15     Pthread_create(&p, NULL, mythread, &args);  
16     Pthread_join(p, (void **) &rvals);  
17     printf("returned %d %d\n", rvals->x, rvals->y);  
18     free(rvals);  
19     return 0;  
20 }
```

Remark: if we are passing a single value (eg:- a long long int) we don't have to package it up as an argument. Instead, we can do as in the following example.

```
void *mythread(void *arg) {  
    long long int value = (long long int) arg;  
    printf("%lld\n", value);  
    return (void *) (value + 1);  
}  
  
int main(int argc, char *argv[]) {  
    pthread_t p;  
    long long int rvalue;  
    Pthread_create(&p, NULL, mythread, (void *) 100);  
    Pthread_join(p, (void **) &rvalue);  
    printf("returned %lld\n", rvalue);  
    return 0;  
}
```

Remark: In all the examples provided above, there is only one thread is created immediately followed by Pthread_join. This is only for illustration purpose. In practice, there will be many threads working together in the same program.

Locks:

Recall that locks are used to prevent multiple threads from entering a critical section of a code simultaneously, thereby ensuring that shared data is accessed in a controlled manner.

Basic Lock operations

- `pthread_mutex_lock(&lock)`

→ acquires the lock. If the lock is already held, the calling thread will block until it becomes available.

- `pthread_mutex_unlock(&lock)`

→ releases the lock. Only the thread with the lock acquired should call unlock.

```
pthread_mutex_t lock;  
pthread_mutex_lock(&lock);  
x = x + 1; // or whatever your critical section is  
pthread_mutex_unlock(&lock);
```

Lock initialization.

Before using a lock, it must be initialized. This can be done statically using `pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER`

or dynamically using `pthread_mutex_t lock;`
`pthread_mutex_init(&lock, NULL);`

(will learn more about locks in the next chapter)

An example

Copy code

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void* mythread(void* arg) {
    pthread_mutex_lock(&lock);
    counter++;
    printf("Thread %d incremented counter to %d\n", *((int*)arg), counter);
    pthread_mutex_unlock(&lock);
    return NULL;
}

int main() {
    pthread_t t[5];
    int id[5];
    for(int i=0;i<5;i++){ id[i]=i+1; pthread_create(&t[i],NULL,mythread,&id[i]); }
    for(int i=0;i<5;i++) pthread_join(t[i],NULL);
    printf("Final counter: %d\n", counter);
    pthread_mutex_destroy(&lock);
    return 0;
}
```